COMP3702 Artificial Intelligence
Semester 2, 2026
Tutorial 2

Before you begin, please note:

- Tutorial exercises are provided to help you understand the materials discussed in class, and to improve your skills in solving AI problems.
- Tutorial exercises will not be graded. However, you are highly encouraged to do them for your own learning. Moreover, we hope you get the satisfaction from solving these problems.
- The skills you acquire in completing the tutorial exercises will help you complete the assignments.
- You'll get the best learning outcome when you try to solve these exercises on your own first (before your tutorial session), and use your tutorial session to ask about the difficulties you face when trying to solve this set of exercises.

Exercises

Exercise 2.1. Implement the Breadth First Search (BFS) and Depth First Search (DFS) algorithms to solve the 8-puzzle problem.
The pseudocode for BFS from the Russell & Norvig textbook (4Ed, using the reached/visited set compared to explored/expanded set for tracking states) is shown in Figure 1. Alternatively, see P&M's discussion of BFS: `https://artint.info/3e/html/ArtInt3e.Ch3.S5.html`.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

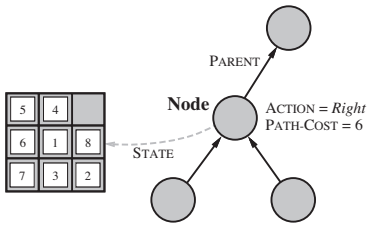

Figure 1: Pseudocode for Breadth-First-Search (left) and corresponding Node representation (right)
Source: Russell & Norvig textbook

You may make use of the supplied `Tutorial_2_template.py` template code which includes a sample EightPuzzle class representing the 8-puzzle states as character strings (e.g. `EightPuzzle("1348627_5")`) and starter code for the nodes in the search graph (i.e. `StateNode`) which efficiently represent paths from initial state to the node's state. You will need to add arguments for the:

- `parent` node,
- `action` taken from parent, and
- `path_cost` (optional but required for frontiers that need to be Priority Queues such as in Uniform Cost Search (UCS), Greedy Best First Search (GBFS), and A* search).

You'll also need to implement a `get_successors` function, and `__eq__`, `__lt__`, and `__hash__` functions to define how elements in the frontier should be compared and sorted.

Exercise 2.2. Test your implementation of BFS and DFS with the following initial and goal states:

- From 1348627_5 to 1238_4765
- From 281_43765 to 1238_4765
- From 281463_75 to 1238_4765

In each test for each algorithm, please output:

(a) The sequence of actions to move from the initial to the goal state.

(b) The number of steps to solve the problem.

(c) The time your implementation takes to solve the problem.

Based on the results of (b) and (c), explain the relation between the time and the number of steps that each algorithm takes to solve the problem. Is the relation you found in-line with the complexity results of the algorithms? If they are not in-line, please find out and explain why.

Exercise 2.3. Suppose there is no solution to the 8-puzzle problem, i.e., there is no way the 8-puzzle can move from the given initial to goal configurations.

(a) Will BFS be able to output the right answer (i.e., no solution)? How about DFS?

(b) We can actually identify if a given 8-puzzle problem is solvable or not without search, which is via the concept of **parity** (see the appended material on inversion and parity). Implement parity computation, so that your program can identify if a problem is solvable without performing search.

Exercise 2.4. Imagine the tiles have been tampered with and it is no longer equally easy to move the tiles in each direction. Suppose that the cost for 'moving the empty tile' is as follows:
cost = {'up':1, 'down':2, 'left':3, 'right':4}

(a) Will Uniform Cost search find the least cost path for this problem?

(b) Compare Uniform Cost Search and BFS when the weight of all edges in the state graph are equal.

Exercise 2.5. Extension Exercise: Complete the same exercises above comparing BFS, DFS and UCS, but on a different environment such as a 2-D maze. Comment on the differences between the algorithms.

Inversion and Parity

We can identify if a given 8-puzzle problem is solvable or not without search by analysing the number of *inversions* between the input state and the goal state. A pair of tiles form an inversion if the values on the tiles are in reverse order of their appearance in the goal state. If the number of inversions is odd it is not solvable; while if it is even it is solvable. This is represented by the *parity*.
To calculate the parity of an 8-puzzle game instance, we first fix the ordering, e.g. left-right, up-down order. We then define the following variables:

- tile-i is the tile with number i on it
- $N(s, i) = \#$inversions of tile-i in state s
  - i.e. the number of out of order pairs of tiles with respect to tile-i
  - This is equal to $\#$tiles with smaller number than tile-i that appears after tile-i
- N(s) is $\#$inversions in state s: $\sum_{i=1}^{n} N(s, i)$
- Parity(s) $= N(s) \bmod 2$
  - Indicates whether the number of inversions of state s is odd or even. If it's even, the parity will be 0 and indicates that the 8-puzzle is solvable

Claim: The parity is *invariant* (does not change) under any legal move of the blank tile.
Proof:

- Any horizontal move will not change the parity.
- Any vertical move changes N by an even number.

Consequence: An 8-puzzle is only solvable if the parity of both the start and goal states are the same.

Example: Odd parity

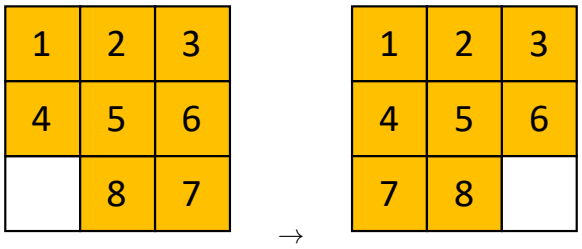

Figure 2: Given state and goal state

For the example in Figure 2, we can write the goal state as 1,2,3,4,5,6,7,8 (ignoring the blank tile) which has even parity (0 inversions), and the initial state as 1,2,3,4,5,6,8,7. In this case, we have 1 inversion in the initial state (7 & 8 are swapped), and therefore its parity is odd. Since the parity of the initial state and goal state are different, the puzzle is not solvable.

Example: Even parity

For the example in Figure 3, we have a total of N(s) = 16 inversions:
N(s,7) = 6;
N(s,2) = 1;
N(s,4) = 2;
N(s,5) = 2;
N(s,6) = 2;
N(s,8) = 2;
N(s,3) = 1;
N(s,1) = 0;

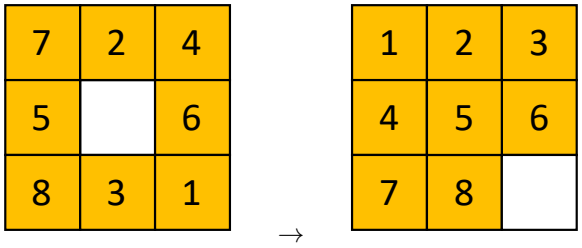

Figure 3: Given state and goal state

Figure 4 shows one way we can count the number of inversions from the "rolled out" representation of the state: '72456831'. For example, for N(s,4) = 2: the '4' tile has 2 numbers after it that are smaller (i.e. the '3' and the '1' which are inverted or in the wrong order relative to the '4' tile).

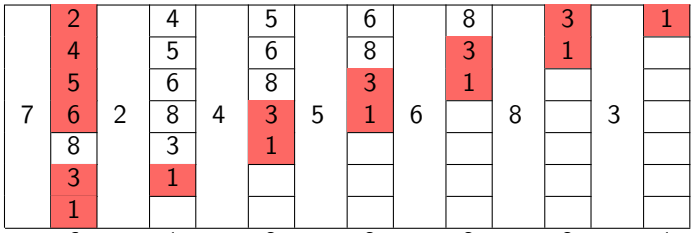

Figure 4: Calculating the number of inversions of each tile for the example in Figure 3. Red cells indicate those tiles that are inverted (in the wrong order) relative to the tile indicated to the left.

Since the parity of the initial and goal states are the same (they are both even), the puzzle is solvable!